

Test Telegram Bot

Testlarga chipta sotish, to'lovni tekshirish va onlayn test topshirishni avtomatlashtiruvchi Telegram bot.

Imkoniyatlari

- 📄 **Yangiliklar** — admin joylagan test e'lonlarini foydalanuvchilar ko'radi
- 📄 **Chipta olish** — test tanlash → F.I.SH → to'lov kartasi → chek yuborish → admin tasdig'i → 6 xonali chipta
- 📄 **Test topshirish** — chipta raqami va F.I.SH orqali kirish, savollarga ketma-ket javob berish, natija va PDF hisobot
- 🧑 **Admin panel** — yangilik/test qo'shish, savollarni kiritish, to'lovlarni tasdiqlash/rad etish, foydalanuvchilar ro'yxati, statistika
- 🗄️ **PostgreSQL** — barcha ma'lumotlar (users, tests, questions, tickets, payments, answers, results, news) saqlanadi
- 📄 **PDF natija** — har bir test tugagach `reportlab` orqali avtomatik hisobot yaratiladi

Texnologiyalar

```
Python 3.12
├─ aiogram          → Telegram bot freymvorki
├─ PostgreSQL       → ma'lumotlar bazasi
├─ asyncpg          → PostgreSQL bilan asinxron aloqa
├─ python-dotenv    → .env orqali maxfiy sozlamalar
└─ reportlab        → PDF generatsiya
```

Loyiha tuzilishi

```
.
├─ bot.py           — botni ishga tushiruvchi asosiy
fayl
├─ config.py        — .env dan sozlamalarni o'qiydi
└─ database.py      — PostgreSQL jadvallari va
```

```

so'rovlar (asynpg)
├─ requirements.txt
├─ .python-version          - 3.12.7
├─ .env.example             - sozlamalar namunasi
├─ .gitignore
├─
├─ handlers/                - foydalanuvchi buyruqlariga
javob beruvchi qism
├─   └─ start.py            - /start, asosiy menyu
├─   └─ news.py            - yangiliklarni ko'rsatish
├─   └─ tickets.py         - chipta sotib olish
jarayoni (FSM)
├─   └─ tests.py           - test topshirish jarayoni
(FSM)
├─   └─ admin.py           - admin panel:
yangilik/test qo'shish, to'lovlar, statistika
├─
├─   └─ services/          - biznes-logika (handlers
bilan database orasidagi qatlam)
├─       └─ ticket_service.py - chipta raqami
generatsiyasi, tekshirish
├─       └─ test_service.py   - test/savol boshqaruvi,
vaqt tekshiruvi
├─       └─ payment_service.py - to'lovni tasdiqlash/rad
etish
├─       └─ pdf_service.py    - natija PDF yaratish

```

O'rnatish (lokal kompyuterda)

1. **Python 3.12 o'rnatish** (loyiha shu versiyada sinovdan o'tgan)

2. **Virtual muhit yaratish:**

```

python3.12 -m venv venv
source venv/bin/activate      # Windows:
venv\Scripts\activate

```

3. **Kutubxonalarni o'rnatish:**

```

pip install -r requirements.txt

```

4. **PostgreSQL bazasini yaratish:**

```
CREATE DATABASE testbot;
```

Jadvallarning o'zi bot birinchi marta ishga tushganda avtomatik yaratiladi — qo'lda SQL yozish shart emas.

5. Sozlamalarni kiriting:

```
cp .env.example .env
```

.env faylni oching va to'ldiring:

O'zgaruvchi	Tavsif
BOT_TOKEN	@BotFather'dan olingan token
ADMIN_IDS	Admin(lar)ning Telegram ID raqami (@userinfobot orqali bilib olinadi), bir nechta bo'lsa vergul bilan: 111,222
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	PostgreSQL ulanish ma'lumotlari
CARD_NUMBER, CARD HOLDER	Foydalanuvchiga ko'rsatiladigan to'lov kartasi ma'lumoti
TICKET_EXPIRE_HOURS	Tasdiqlangan chiptaning amal qilish muddati (soatda)

6. Botni ishga tushiring:

```
python bot.py
```

Render'da deploy qilish

- Loyihani GitHub repozitoriyasiga yuklang — **fayllar repo tugida to'g'ridan-to'g'ri turishi kerak** (bot.py repo bosh sahifasida ko'rinishi kerak, alohida ichki papkada emas).
- Render'da **New** → **Background Worker** yarating (bot polling rejimida ishlagani uchun Web Service emas, aynan Background Worker tanlanadi).
- Sozlamalar:

- **Root Directory:** bo'sh qoldiring (agar fayllar repo tugida bo'lsa)
 - **Build Command:** `pip install -r requirements.txt`
 - **Start Command:** `python bot.py`
4. **Environment** bo'limiga `.env` dagi barcha o'zgaruvchilarni qo'shing.
 5. Render'da PostgreSQL instansiyasi yarating (yoki tashqi bazadan foydalaning) va uning ulanish ma'lumotlarini shu yerga kiriting.
 6. Deploy qiling. Agar avvalgi urinishlarda xatoliklar bo'lgan bo'lsa, **Manual Deploy** → **Clear build cache & deploy** orqali eski keshni tozalab qayta urining.

⚠ Uchrashi mumkin bo'lgan xatoliklar

Xato	Sabab	Yechim
<code>ModuleNotFoundError: No module named 'aiogram'</code>	<code>requirements.txt</code> topilmagan yoki build bosqichi ishlamagan	Root Directory sozlamasini tekshiring, Build Command'ni tasdiqlang, build log'ni to'liq ko'ring
<code>Root directory "... " does not exist</code>	Render Settings'dagi Root Directory GitHub tuzilishiga mos kelmayapti	Fayllar repo tugida bo'lsa, Root Directory'ni bo'sh qoldiring
<code>gcc ... asynpcpg/pgproto/pgproto.c kompilyatsiya xatosi</code>	<code>asynpcpg</code> versiyasi ishlatilayotgan Python versiyasi uchun tayyor wheel'ga ega emas	<code>requirements.txt</code> da <code>asynpcpg==0.31.0</code> ishlatilg va <code>.python-version</code> faylida barqaror versiyani (<code>3.12.7</code>) belgilang

📌 Eslatmalar

- Test vaqti tugashi (`duration`) har bir javobdan keyin tekshiriladi. Agar foydalanuvchi javob bermay botni yopib qo'ysa, sessiya keyingi harakatgacha (yoki " `Testdan chiqish`" tugmasigacha) yakunlanmaydi — qat'iy vaqt chegarasi kerak bo'lsa, alohida rejalashtiruvchi (masalan APScheduler) qo'shish mumkin.

- `.env` fayl hech qachon reponi ochiq (public) qilmang — u `.gitignore` da allaqachon istisno qilingan.